Python – Formelsamling

Teknologi- og dataintegration · E25AB

1. Datatyper

Type	Betyder	Eksempel
`str`	Tekst (streng)	`"Sara"`, `"12"`
`int`	Heltal	`21`, `-5`, `0`
`float`	Decimaltal	`3.14`, `2.0`
`bool`	Sandhedsværdi	`True`, `False`

`type(x)` fortæller dig, hvilken datatype `x` har.

2. Konvertering mellem datatyper

Funktion	Gør
`int(x)`	Laver x om til et heltal – **kapper** decimaler, runder ikke
`float(x)`	Laver x om til et decimaltal
`str(x)`	Laver x om til tekst

Pas på!

- `"12" + "8"` giver `"128"` (tekst sættes sammen), ikke `20`.
- `int(9.9)` giver `9` – ikke `10`. `int()` runder ikke, den fjerner bare alt efter kommaet.
- `"20" >= 18` giver **fejl** – man kan ikke sammenligne tekst (`str`) og tal (`int`) direkte.
- Tjek altid datatypen, hvis noget virker mærkeligt: `type(x)`.

3. Lister

En liste skrives med `[ ]` og indeholder elementer adskilt af komma. Det **første** element har indeks `0`.

```
1  frugter = ["æble", "banan", "pære"]
2  print(frugter[0])   # "æble" (det første)
3  print(frugter[-1])  # "pære" (det sidste)
```

Typiske liste-funktioner

Funktion	Gør
`len(liste)`	Antal elementer i listen
`liste[i]`	Elementet på plads `i` (indeks) – første plads er `0`
`liste[-1]`	Det sidste element i listen
`liste.append(x)`	Tilføjer x sidst i listen
`for x in liste:`	Looper igennem hvert element i listen, ét ad gangen
`for i, x in enumerate(liste):`	Looper igennem listen og giver både indeks `i` og element `x`
`while i < len(liste):`	Looper med et tælleindeks `i` – bruges ofte sammen med `liste[i]`

4. Regneoperatorer

Tegn	Navn	Eksempel	Resultat
`+`	Plus	`5 + 3`	`8`
`-`	Minus	`5 - 3`	`2`
`*`	Gange	`5 * 3`	`15`
`/`	Division	`9 / 4`	`2.25`
`//`	Heltalsdivision (runder **ned**)	`9 // 4`	`2`
`%`	Modulo (resten ved division)	`9 % 4`	`1`

Modulo bruges tit til at tjekke lige/ulige tal: `tal % 2 == 0` er sandt, hvis `tal` er lige.

5. Sammenligningsoperatorer

Tegn	Betyder
`==`	Er lig med
`!=`	Er **ikke** lig med
`>`	Større end
`<`	Mindre end
`>=`	Større end eller lig med
`<=`	Mindre end eller lig med

Resultatet af en sammenligning er altid `True` eller `False` (`bool`).

6. Boolske operatorer (and / or / not)

Operator	Betyder	Sand hvis...
`and`	Og	**begge** betingelser er sande
`or`	Eller	**mindst én** betingelse er sand
`not`	Ikke	vender resultatet om (`True` bliver `False`)

`True` tæller som `1` og `False` tæller som `0`, når de bruges sammen med `+` eller `-`:

```
1  pris = 100 + True   # pris bliver 101
2  pris = 100 - False  # pris bliver 100
```

- En flag-variabel (fx `fundet = False`) starter typisk som `False` og skiftes til `True`, når noget bestemt sker i koden.

7. If / elif / else

if / elif / else

Computeren tjekker betingelserne **oppefra og ned, én ad gangen**. Så snart én betingelse er sand:

- Den tilhørende gren køres.
- Resten af `elif` / `else` bliver **ikke** tjekket – heller ikke selvom de også ville have været sande.

```
karakter = 7

if karakter >= 10:      # False → videre
    print("Fremragende")
elif karakter >= 7:     # True → STOP her
    print("Godt")
elif karakter >= 4:     # springes over
    print("Jævnt")
else:                   # springes over
    print("Dumpet")
```

Output: `Godt`

Flere separate if-sætninger (uden elif)

Denne gang hører linje 3-6 sammen som et `if`/`else`-par – kun én af dem kan blive kørt. Linje 7-10 er derimod to **selvstændige** `if`-sætninger, som bliver tjekket helt uafhængigt af if/else-parret ovenfor (og af hinanden):

```
karakter = 7

if karakter >= 10:      # False → else
    print("Fremragende")
else:                   # køres (if var falsk)
    print("Ikke fremragende")
if karakter >= 4:       # selvstændig if
    print("Jævnt")
if karakter < 4:        # False, springes over
    print("Dumpet")
```

Output:

```
Ikke fremragende
Jævnt
```

Der bliver stadig printet **to** linjer: én fra if/else-parret (linje 3-6) og én fra den selvstændige if på linje 7. If/else-parret opfører sig som i afsnittet ovenfor – kun én gren køres – men de efterfølgende if-sætninger er ikke en del af det par, så de bliver tjekket, uanset hvad if/else endte med at gøre.

Sammenligning

	`if / elif / else`	`if/else` + selvstændige `if`
Hvor mange betingelser bliver tjekket?	Stopper ved den **første** sande	Hvert `if/elif/else`-**par** stopper for sig – men de næste, selvstændige `if`-sætninger bliver altid tjekket
Hvor mange grene kan blive kørt?	Højst **én** i alt	Højst én **pr. if/elif/else-par**, men flere samlet set

8. Løkker (loops)

for-loop med `range(start, stop, step)`

Tæller fra `start` op til (men **ikke med**) `stop`, i spring på `step` ad gangen. `step` er `1`, hvis den ikke skrives.

Kode	Værdi af t
`for t in range(5):`	`0 1 2 3 4`
`for t in range(1, 5):`	`1 2 3 4`
`for t in range(2, 11, 2):`	`2 4 6 8 10`

for-loop med `enumerate(liste)`

Giver **både indeks og element** i hver omgang, så du slipper for selv at tælle med. Indekset starter ved `0`. `enumerate(liste, 1)` starter i stedet ved `1`.

```
farver = ["rød", "grøn", "blå"]
for i, farve in enumerate(farver):
    print(i, farve)
# 0 rød
# 1 grøn
# 2 blå
```

while-loop

Kører, **så længe** betingelsen er sand. Husk at ændre variablen inde i loopet, ellers kører det for evigt.

```
tal = 5
while tal > 0:
    print(tal)
    tal = tal - 1
# tal: 5, 4, 3, 2, 1
```

break

Stopper loopet med det samme, uanset betingelsen. I et nested loop (loop i loop) stopper `break` kun det **inderste** loop – det ydre loop fortsætter.

Loop i loop (nested loop)

For hver værdi i det ydre loop kører **hele** det indre loop igennem, før det ydre skifter til næste værdi.

```
for i in range(1, 3):
    for j in range(1, 3):
        print(i, j)
# (1,1) (1,2) (2,1) (2,2)
```

9. Strings (tekst) som en liste af bogstaver

Et for-loop kan gå igennem bogstaverne i en tekst, ét ad gangen. `len(tekst)` tæller antal bogstaver.

```
for bogstav in "kode":
    print(bogstav)
# k
# o
# d
# e
```

10. Sådan arbejder du med kode på papir

- Koden køres **altid** oppefra og ned, linje for linje.
- En variabel husker sin værdi, indtil en ny linje ændrer den.
- Lav en lille tabel med én kolonne pr. variabel, og opdater den for hver omgang i et loop.
- Ved nested loops: brug to kolonner (fx `i` og `j`), og husk at det indre loop kører helt færdigt, før det ydre tæller videre.